📋 Скрипт презентации проекта DevOps

Автор: Vitalii Zaburdaiev
Курс: DevOpsUA6
Проект: Health Dashboard - DevOps Implementation
Длительность: 10-15 минут (базовая версия)

🎯 1. ВВЕДЕНИЕ (30 секунд)

Что говорить:

> "Добрый день! Меня зовут Виталий Забурдаев, я студент курса DevOpsUA6.
> Сегодня я представлю вам свой выпускной проект - **Health Dashboard** - веб-приложение с полноценной DevOps инфраструктурой.
>
> В проекте я реализовал полный цикл: от разработки приложения до его развертывания в облаке с автоматизацией, мониторингом и обеспечением безопасности.
>
> Презентация займет примерно 12-15 минут, после чего я с удовольствием отвечу на ваши вопросы."

Советы:

- Говорите уверенно и спокойно
- Улыбнитесь в начале
- Установите зрительный контакт
- Проверьте, что экран виден всем

📊 2. СТРУКТУРА ПРЕЗЕНТАЦИИ ПО СЛАЙДАМ

СЛАЙД 1: Титульный слайд ⏱️ 30 сек

Что показано:

- Название проекта: Health Dashboard
- Ваше имя: Vitalii Zaburdaiev
- Курс: DevOpsUA6
- Возможно логотип или скриншот приложения

Что говорить:

> "Мой проект называется **Health Dashboard** - это веб-приложение для мониторинга здоровья системы.
> Я прошёл полный путь от написания кода до развертывания production-ready инфраструктуры в облаке AWS.
> Давайте посмотрим, что удалось реализовать."

Ключевые моменты:

- Название запоминающееся
- Проект реальный, работающий
- Полный DevOps цикл

СЛАЙД 2: Обзор проекта ⏱️ 1 минута

Что показано:

- Скриншот работающего приложения
- Основные функции
- Технологический стек

Что говорить:

> "**Health Dashboard** - это Flask-приложение, которое отображает статистику здоровья пользователей.
>
> Приложение включает:
> - **Frontend** на HTML/CSS с адаптивным дизайном
> - **Backend** на Python Flask с REST API
> - **База данных PostgreSQL** для хранения данных пользователей
> - **Redis** для кэширования и сессий
>
> Но главная ценность проекта - это не само приложение, а **DevOps инфраструктура**, которую я вокруг него построил.
> Приложение работает в production на AWS, доступно по адресу **3.127.155.114**, и весь процесс от commit до deploy полностью автоматизирован."

Ключевые моменты:

- Приложение РАБОТАЕТ (покажите на живом сервере)
- Не просто код, а полная инфраструктура
- Production-ready решение

Технические детали (если спросят):

- Flask 2.x с Blueprints
- PostgreSQL 14
- Redis 7
- Nginx как reverse proxy
- 12 unit тестов с coverage 85%+

СЛАЙД 3: Архитектура решения ⏱️ 1.5 минуты

Что показано:

- Диаграмма архитектуры
- Компоненты системы
- Потоки данных

Что говорить:

> “Давайте посмотрим на архитектуру решения.
>
> **Клиентская часть:**
> Пользователь обращается к приложению через браузер. Запросы приходят на **AWS EC2 сервер**.
>
> **Сетевой уровень:**
> Первой точкой входа является **Nginx**, который работает как reverse proxy. Он принимает HTTPS запросы, терминирует SSL и направляет трафик на Flask приложение.
>
> **Уровень приложения:**
> **Flask приложение** обрабатывает бизнес-логику. Для ускорения работы используется **Redis** - часто запрашиваемые данные кэшируются, что снижает нагрузку на базу данных.
>
> **Уровень данных:**
> Основные данные хранятся в **PostgreSQL** - надежной реляционной базе данных.
>
> **Мониторинг:**
> Все сервисы отдают метрики в **Prometheus**, который собирает их каждые 15 секунд. **Grafana** визуализирует эти метрики в красивых дашбордах.
>
> Все эти компоненты работают в **Docker контейнерах**, управляемых через **Docker Compose** - всего 7 сервисов в одной оркестрации.”

Ключевые моменты:

- Многослойная архитектура
- Разделение ответственности
- Масштабируемость заложена изначально
- Мониторинг - часть архитектуры, не afterthought

Технические детали (если спросят):

- Nginx на порту 80 (можно добавить 443 для SSL)
- Flask на порту 5000 (внутри сети Docker)
- PostgreSQL на порту 5432
- Redis на порту 6379
- Prometheus на 9090
- Grafana на 3000
- Node Exporter для метрик сервера

СЛАЙД 4: Docker и контейнеризация ⏱️ 1.5 минуты

Что показано:

- Список Docker сервисов
- Структура Docker Compose
- Преимущества контейнеризации

Что говорить:

> “Весь проект контейнеризирован с помощью **Docker**.
>
> Я создал **Dockerfile** для Flask приложения, который:
> - Использует базовый образ Python 3.9-slim для минимального размера
> - Устанавливает все зависимости из requirements.txt
> - Копирует код приложения
> - Настраивает правильные permissions
> - Запускает приложение через Gunicorn для production
>
> Для оркестрации всех сервисов используется **Docker Compose**, который управляет 7 контейнерами:
> 1. **web** - Flask приложение
> 2. **db** - PostgreSQL база данных
> 3. **redis** - кэш и хранилище сессий
> 4. **nginx** - reverse proxy
> 5. **prometheus** - сбор метрик
> 6. **grafana** - визуализация метрик
> 7. **node-exporter** - метрики сервера
>
> **Преимущества такого подхода:**
> - Полная изоляция сервисов
> - Воспроизводимость окружения (dev = staging = prod)
> - Легкое масштабирование
> - Быстрый rollback при проблемах
> - Один файл docker-compose.yml описывает всю инфраструктуру”

Ключевые моменты:

- Multi-stage builds для оптимизации размера
- Docker networks для изоляции
- Volumes для персистентности данных
- Health checks для всех сервисов

Технические детали (если спросят):

- Образы с тегами для версионирования
- .dockerignore для оптимизации build context
- Docker secrets для sensitive данных (можно улучшить)
- Resource limits в docker-compose (память, CPU)

СЛАЙД 5: CI/CD Pipeline ⏱️ 2 минуты

Что показано:

- Схема GitHub Actions pipeline
- Этапы CI/CD
- Скриншот успешного workflow

Что говорить:

> “Один из ключевых элементов DevOps - это **автоматизация**. Я настроил полноценный CI/CD pipeline с помощью **GitHub Actions**.
>
> **Вот как это работает:**
>
> **Триггер:**
> Когда я делаю `git push` в ветку `main`, автоматически запускается pipeline.
>
> **Этап 1: Continuous Integration (CI)**
> 1. **Checkout кода** - GitHub Actions клонирует репозиторий
> 2. **Запуск тестов** - выполняются все 12 unit тестов с помощью pytest
> 3. **Проверка качества кода** - линтеры проверяют код style (flake8, pylint)
> 4. **Сборка Docker образа** - если тесты прошли, собирается новый образ
> 5. **Push в Docker Hub** - образ загружается в registry с тегом версии
>
> **Этап 2: Continuous Deployment (CD)**
> 6. **SSH подключение к AWS серверу** - через защищенные secrets
> 7. **Pull нового образа** - сервер скачивает свежий образ из Docker Hub
> 8. **Обновление сервисов** - docker-compose выполняет rolling update
> 9. **Health check** - проверка, что приложение поднялось корректно
> 10. **Уведомление** - в случае успеха или ошибки (можно добавить Slack/Email)
>
> **Результат:**
> От коммита до production развертывания проходит **около 5 минут**. Полностью автоматически. Без ручного вмешательства.
>
> Все успешные запуски можно увидеть в GitHub Actions - зеленые галочки подтверждают, что код рабочий.”

Ключевые моменты:

- **Zero-downtime deployment** - пользователи не замечают обновление
- **Автоматические rollback** - если тесты не прошли, deploy не происходит
- **Версионирование** - каждый образ имеет тег с commit hash
- **Безопасность** - SSH ключи и пароли хранятся в GitHub Secrets

Технические детали (если спросят):

- GitHub Actions workflow файл: `.github/workflows/ci-cd.yml`
- Используется self-hosted runner или GitHub-hosted
- Кэширование зависимостей для ускорения
- Matrix builds для тестирования на разных версиях Python
- Можно добавить staging окружение перед production

СЛАЙД 6: Kubernetes оркестрация ⏱️ 1.5 минуты

Что показано:

- Схема Kubernetes кластера
- Основные объекты K8s
- Преимущества Kubernetes

Что говорить:

> “Для продвинутой оркестрации контейнеров я внедрил **Kubernetes**.
>
> **Что настроено:**
>
> **Deployments:**
> Я создал Kubernetes манифесты для каждого компонента - Flask приложения, PostgreSQL, Redis. Каждый Deployment описывает желаемое состояние: сколько реплик должно работать, какой образ использовать, какие ресурсы выделить.
>
> **Services:**
> Настроены Service объекты, которые обеспечивают стабильные сетевые endpoints. Даже если pod перезапускается и получает новый IP, Service продолжает работать.
>
> **ConfigMaps и Secrets:**
> Конфигурация вынесена в ConfigMaps - переменные окружения, настройки приложения. Чувствительные данные (пароли БД, API ключи) хранятся в Secrets в зашифрованном виде.
>
> **Helm Charts:**
> Для упрощения деплоя я использовал **Helm** - пакетный менеджер для Kubernetes. Один команда `helm install` разворачивает всё приложение со всеми зависимостями.
>
> **Зачем это нужно?**
> - **Самовосстановление** - если контейнер падает, K8s автоматически перезапускает
> - **Автомасштабирование** - при росте нагрузки поднимаются дополнительные реплики
> - **Rolling updates** - обновления без простоя
> - **Load balancing** - трафик равномерно распределяется между репликами
>
> Это production-grade решение, которое используют крупнейшие компании мира.”

Ключевые моменты:

- Kubernetes - стандарт индустрии
- Готовность к scale
- Высокая доступность (High Availability)

Технические детали (если спросят):

- Используется minikube/k3s для локальной разработки или AWS EKS
- Настроены health checks (liveness, readiness probes)
- Resource limits и requests
- Persistent Volumes для баз данных
- Ingress для маршрутизации внешнего трафика

СЛАЙД 7: Terraform - Infrastructure as Code ⏱️ 1.5 минуты

Что показано:

- Схема AWS инфраструктуры
- Terraform код
- Созданные ресурсы

Что говорить:

> “Для управления облачной инфраструктурой я использовал **Terraform** - инструмент Infrastructure as Code.
>
> **Что это значит?**
> Вся инфраструктура в AWS описана в виде кода. Вместо того, чтобы вручную создавать серверы через веб-интерфейс AWS, я написал Terraform конфигурацию, которая автоматически:
>
> **Создаёт AWS ресурсы:**
> - **EC2 инстанс** - виртуальный сервер (тип t2.medium, Ubuntu 22.04)
> - **Security Groups** - правила файрвола (открыты порты 22, 80, 443, 3000, 9090)
> - **Elastic IP** - статический IP адрес (3.127.155.114)
> - **VPC и подсети** - сетевая изоляция
> - **SSH ключи** - для безопасного доступа
>
> **Преимущества:**
> - **Воспроизводимость** - одной командой `terraform apply` я могу развернуть идентичную инфраструктуру
> - **Версионирование** - инфраструктура хранится в Git, все изменения отслеживаются
> - **Откат изменений** - можно вернуться к предыдущей версии
> - **Документация** - код Terraform сам документирует архитектуру
> - **Планирование** - `terraform plan` показывает, что изменится ДО применения
>
> Это критически важно для production - никаких ручных изменений, всё через код и Git.”

Ключевые моменты:

- Infrastructure as Code - best practice
- Terraform state хранится безопасно
- Модульная структура кода
- Multi-environment support (dev, staging, prod)

Технические детали (если спросят):

- AWS Provider версия
- Используется remote state в S3 (или локально для учебного проекта)
- State locking с DynamoDB
- Модули для переиспользования кода
- Variables для параметризации
- Outputs для получения важных значений (IP, DNS)

СЛАЙД 8: Ansible - автоматизация конфигурации ⏱️ 1 минута

Что показано:

- Ansible playbooks
- Задачи конфигурации
- Роли и структура

Что говорить:

> "После создания сервера с Terraform, его нужно настроить. Для этого я использовал **Ansible**.
>
> **Что делает Ansible:**
>
> Я написал Ansible playbook, который автоматически:
> 1. **Обновляет систему** - `apt update && apt upgrade`
> 2. **Устанавливает необходимое ПО** - Docker, Docker Compose, Git, monitoring tools
> 3. **Настраивает пользователей** - создаёт deploy user с правильными правами
> 4. **Конфигурирует файрвол** - ufw rules для безопасности
> 5. **Копирует конфигурационные файлы** - nginx.conf, prometheus.yml и т.д.
> 6. **Запускает сервисы** - docker-compose up
> 7. **Настраивает мониторинг** - node exporter, log rotation
>
> **Преимущества:**
> - **Идемпотентность** - можно запускать многократно, результат один и тот же
> - **Декларативный подход** - описываем желаемое состояние, а не шаги
> - **Безопасность** - нет необходимости вручную заходить на сервер
> - **Масштабирование** - один playbook может настроить 100 серверов одновременно
>
> Один запуск `ansible-playbook deploy.yml` и сервер полностью готов к работе."

Ключевые моменты:

- Автоматизация настройки сервера
- Повторяемые деплои
- Inventory для управления множеством серверов

Технические детали (если спросят):

- Ansible roles для модульности
- Ansible Vault для секретов
- Handlers для перезапуска сервисов
- Tags для выборочного выполнения задач
- Проверка с `--check` и `--diff` перед применением

СЛАЙД 9: Мониторинг - Prometheus и Grafana ⏱️ 1.5 минуты

Что показано:

- Скриншот Grafana дашборда
- Графики метрик
- Алерты

Что говорить:

> “Невозможно управлять тем, что не измеряешь. Поэтому я настроил полноценный **мониторинг** с помощью Prometheus и Grafana.
>
> **Prometheus** собирает метрики:
> - **Метрики приложения** - количество HTTP запросов, время ответа, ошибки
> - **Метрики инфраструктуры** - CPU, память, диск, сеть (через node-exporter)
> - **Метрики баз данных** - количество соединений к PostgreSQL, размер БД
> - **Метрики Docker** - состояние контейнеров, использование ресурсов
>
> Prometheus опрашивает endpoints каждые 15 секунд и хранит временные ряды.
>
> **Grafana** визуализирует эти данные в красивых дашбордах:
> - **System Overview** - общее здоровье системы (CPU, RAM, Disk, Network)
> - **Application Performance** - метрики Flask приложения
> - **Database Dashboard** - состояние PostgreSQL
> - **Docker Containers** - статус всех контейнеров
>
> **Настроены алерты:**
> - CPU выше 80% - предупреждение
> - Память выше 90% - критический алерт
> - Приложение не отвечает - немедленное уведомление
> - Диск заполнен более чем на 85% - warning
>
> Давайте посмотрим на живой дашборд...”

[Здесь можно переключиться на браузер и показать Grafana по адресу 3.127.155.114:3000]

Ключевые моменты:

- Проактивный мониторинг, а не реактивный
- Метрики помогают оптимизировать производительность
- Алерты предотвращают downtime

Технические детали (если спросят):

- PromQL для запросов метрик
- Retention policy для данных (обычно 15 дней)
- Alertmanager для маршрутизации алертов
- Интеграция с Slack/Email/PagerDuty для уведомлений
- Custom metrics из приложения через Prometheus client library

---

СЛАЙД 10: Безопасность ⏱️ 1 минута

Что показано:

- Чек-лист мер безопасности
- Схема защиты
- Security best practices

Что говорить:

> “Безопасность была приоритетом с первого дня разработки.
>
> **Реализованные меры:**
>
> **Сетевая безопасность:**
> - **Файрвол** - AWS Security Groups разрешают только необходимые порты
> - **SSH доступ** - только по ключу, пароли отключены
> - **Fail2ban** - блокировка после нескольких неудачных попыток входа
>
> **Безопасность приложения:**
> - **Environment variables** - никаких хардкод паролей в коде
> - **Secrets management** - чувствительные данные в GitHub Secrets и K8s Secrets
> - **SQL Injection защита** - ORM (SQLAlchemy) предотвращает инъекции
> - **CSRF токены** - защита от межсайтовой подделки запросов
>
> **Безопасность контейнеров:**
> - **Non-root пользователи** - контейнеры работают не от root
> - **Read-only файловые системы** где возможно
> - **Сканирование образов** - проверка на уязвимости (можно добавить Trivy)
>
> **Безопасность данных:**
> - **Бэкапы базы данных** - автоматические snapshot каждый день
> - **Логирование** - все действия записываются в журналы
>
> **SSL/TLS:**
> - Можно добавить Let’s Encrypt для HTTPS (следующий шаг)
>
> Безопасность - это не одноразовая настройка, а постоянный процесс.”

Ключевые моменты:

- Defense in depth - многоуровневая защита
- Принцип least privilege
- Регулярные обновления

Технические детали (если спросят):

- Security Groups правила (ingress/egress)
- IAM roles для AWS ресурсов
- Docker security best practices
- OWASP Top 10 учтены
- План incident response

СЛАЙД 11: Проблемы и решения ⏱️ 1.5 минуты

Что показано:

- Список challenges
- Решения
- Уроки, которые вынесли

Что говорить:

> “Конечно, не всё шло гладко. Позвольте поделиться некоторыми вызовами и тем, как я их решил.
>
> **Проблема 1: Docker networking**
> **Вызов:** Flask приложение не могло подключиться к PostgreSQL.
> **Решение:** Изучил Docker networks, создал отдельную сеть для всех сервисов, использовал service names как DNS имена. Теперь контейнеры общаются между собой по именам.
>
> **Проблема 2: CI/CD pipeline падал на тестах**
> **Вызов:** Тесты проходили локально, но падали в GitHub Actions.
> **Решение:** Обнаружил, что не хватало переменных окружения. Добавил `.env.test` файл и настроил GitHub Secrets. Научился важности идентичности окружений.
>
> **Проблема 3: Высокое потребление памяти на AWS**
> **Вызов:** Сервер часто исчерпывал память, приложение падало.
> **Решение:** Добавил resource limits в docker-compose, настроил swap, оптимизировал Gunicorn workers. Научился важности мониторинга - Grafana показала проблему.
>
> **Проблема 4: Долгий deploy**
> **Вызов:** Каждый deploy занимал 10+ минут из-за пересборки всех образов.
> **Решение:** Внедрил multi-stage Docker builds, настроил кэширование слоёв, использовал .dockerignore. Время сократилось до 3-5 минут.
>
> **Проблема 5: Потеря данных при перезапуске контейнеров**
> **Вызов:** После `docker-compose down` терялись данные БД.
> **Решение:** Настроил Docker volumes для персистентности. Теперь данные сохраняются независимо от жизненного цикла контейнеров.
>
> **Главный урок:**
> DevOps - это итеративный процесс. Каждая проблема научила чему-то важному. Документирование решений помогло не повторять ошибки.”

Ключевые моменты:

- Честность о challenges показывает зрелость
- Решение проблем - ценный опыт
- Continuous improvement mindset

Технические детали (если спросят):

- Конкретные error messages и как их дебажили
- Использованные инструменты для troubleshooting
- Как использовали логи для диагностики

СЛАЙД 12: Заключение и результаты ⏱️ 1 минута

Что показано:

- Итоги проекта
- Достигнутые результаты
- Планы на будущее

Что говорить:

> “Подводя итоги, что удалось реализовать:
>
> **Технические достижения:**
> ✅ Разработано и развернуто production-ready веб-приложение
> ✅ Настроена полная CI/CD автоматизация - от commit до deploy за 5 минут
> ✅ Инфраструктура в AWS управляется как код через Terraform
> ✅ Все сервисы контейнеризированы - 7 Docker контейнеров
> ✅ Kubernetes оркестрация для масштабируемости
> ✅ Автоматизация конфигурации с Ansible
> ✅ Полноценный мониторинг - Prometheus + Grafana с алертами
> ✅ 12 unit тестов с хорошим покрытием
> ✅ Безопасность на всех уровнях
>
> **Главное достижение:**
> Я создал не просто приложение, а **полноценную DevOps экосистему**, которая отражает реальные практики индустрии.
>
> **Навыки, которые освоил:**
> - Infrastructure as Code
> - Контейнеризация и оркестрация
> - CI/CD pipelines
> - Cloud computing (AWS)
> - Мониторинг и логирование
> - Автоматизация
> - Security best practices
>
> **Планы на будущее:**
> - Добавить HTTPS с Let’s Encrypt
> - Внедрить централизованное логирование (ELK stack)
> - Настроить автомасштабирование в K8s
> - Добавить performance тесты
> - Multi-region deployment для высокой доступности
>
> **Проект доступен:**
> - 🌐 Приложение: http://3.127.155.114
> - 📊 Grafana: http://3.127.155.114:3000
> - 💻 GitHub: https://github.com/zaburdaev/my-devops-project
>
> Спасибо за внимание! Готов ответить на ваши вопросы.”

Ключевые моменты:

- Подчеркните масштаб работы
- Покажите enthusiasm о проекте
- Пригласите к вопросам

🎬 3. ДЕМОНСТРАЦИЯ ПРОЕКТА (5 минут)

Сценарий live demo:

Шаг 1: Показать работающее приложение (1 мин)

1. Откройте браузер
2. Перейдите на **http://3.127.155.114**
3. Покажите главную страницу
4. Продемонстрируйте функционал (добавление данных, просмотр статистики)

Что говорить:

> "Вот само приложение в действии. Оно работает на AWS сервере. Давайте добавим несколько записей… Видите, данные сохраняются в PostgreSQL, интерфейс отзывчивый благодаря Redis кэшированию."

Шаг 2: Показать Grafana мониторинг (1.5 мин)

1. Откройте новую вкладку: **http://3.127.155.114:3000**
2. Войдите в Grafana (admin/admin или ваши credentials)
3. Покажите дашборды:
   - System metrics (CPU, RAM, Disk)
   - Application metrics
   - Docker containers status

Что говорить:

> "Теперь посмотрим на мониторинг. Здесь в реальном времени видно:
> - CPU использование - сейчас около 25%
> - Память - 60% занято
> - Сеть - входящий и исходящий трафик
> - Количество HTTP запросов к приложению
> - Все Docker контейнеры работают (зелёные индикаторы)
>
> Если что-то пойдёт не так, я получу алерт немедленно."

Шаг 3: Показать GitHub Actions (1 мин)

1. Откройте **https://github.com/zaburdaev/my-devops-project**
2. Перейдите во вкладку **Actions**
3. Покажите последний успешный workflow run
4. Откройте детали - покажите этапы CI/CD

Что говорить:

> "Вот история всех деплоев. Зелёные галочки - успешные builds. Давайте посмотрим на последний…
> - Тесты прошли - все 12 passed
> - Docker образ собран
> - Развёрнут на сервере
> - Всё за 4 минуты 32 секунды
>
> Если я сейчас сделаю изменение в коде и push, через 5 минут это будет в production."

Шаг 4: Показать SSH подключение и Docker (1 мин)

1. Откройте терминал
2. SSH на сервер: `ssh ubuntu@3.127.155.114`
3. Выполните команды:
   ```bash
   docker ps  # показать запущенные контейнеры
   docker-compose ps  # статус сервисов
   docker stats --no-stream  # использование ресурсов
   ```

Что говорить:

> "Давайте заглянем на сам сервер. Подключаюсь по SSH...
>
> Команда `docker ps` показывает все запущенные контейнеры - вот они, все 7 сервисов работают.
>
> `docker stats` показывает потребление ресурсов каждым контейнером в реальном времени."

Шаг 5: Показать Infrastructure Code (30 сек)

1. Вернитесь в GitHub
2. Покажите структуру репозитория:
   - `terraform/` - инфраструктура
   - `ansible/` - конфигурация
   - `k8s/` - Kubernetes манифесты
   - `docker-compose.yml` - оркестрация
   - `.github/workflows/` - CI/CD

Что говорить:

> "Вся инфраструктура хранится в Git. Каждая папка - отдельный компонент DevOps пайплайна. Всё версионируется, всё воспроизводимо."

❓ 4. ОТВЕТЫ НА ВОЗМОЖНЫЕ ВОПРОСЫ

Q1: Почему выбрали именно такую архитектуру?

Ответ:

> “Я выбрал микросервисную архитектуру с разделением на отдельные компоненты по нескольким причинам:
>
> 1. **Масштабируемость** - каждый сервис можно масштабировать независимо. Если нагрузка на приложение растёт, я могу поднять больше реплик Flask, не трогая базу данных.
> 2. **Изоляция** - сбой в одном контейнере не уронит всю систему. PostgreSQL работает независимо от Redis.
> 3. **Flexibility** - легко заменить компонент. Например, если захочу сменить Redis на Memcached, это не затронет остальные части.
> 4. **DevOps best practices** - эта архитектура отражает то, как строят системы в реальных компаниях. Монолит проще для маленького проекта, но я хотел освоить production подходы.
> 5. **Мониторинг** - с отдельными сервисами проще отслеживать, где проблема. Grafana показывает метрики каждого компонента отдельно.”

Q2: Как именно работает ваш CI/CD pipeline? Опишите подробнее.

Ответ:

> “Конечно! Вот детальный flow:
>
> **Триггер:** Я делаю `git push` в ветку main.
>
> **GitHub Actions получает событие** и запускает workflow, описанный в `.github/workflows/ci-cd.yml`.
>
> **Job 1: Test & Build (CI)**
> - Runner создаёт чистое Ubuntu окружение
> - Клонирует код из репозитория
> - Устанавливает Python и зависимости
> - Запускает pytest - все 12 тестов должны пройти
> - Если хотя бы один тест упал - pipeline останавливается, deploy не происходит
> - Запускает линтеры (flake8, black) - проверка code style
> - Если всё OK - собирает Docker образ с тегом `latest` и `<commit-sha>`
> - Логинится в Docker Hub используя secrets
> - Пушит образ в registry
>
> **Job 2: Deploy (CD)**
> - Подключается к AWS EC2 серверу по SSH (ключ хранится в GitHub Secrets)
> - Выполняет команды на удалённом сервере:
> - `docker-compose pull` - скачивает новый образ
> - `docker-compose up -d` - пересоздаёт контейнеры с новым образом
> - Старые контейнеры gracefully останавливаются
> - Новые поднимаются - zero downtime благодаря rolling update
> - Health check - проверка, что приложение отвечает на /health endpoint
> - Если health check failed - автоматический rollback к предыдущей версии
>
> **Результат:** От коммита до production - полностью автоматизировано.”

Q3: Что произойдёт, если сервер упадёт? Какой у вас disaster recovery план?

Ответ:

> “Отличный вопрос про resilience.
>
> **Текущая ситуация:**
> Сейчас у меня single EC2 instance, поэтому если сервер упадёт:
> - Приложение станет недоступно до восстановления
> - Но данные НЕ потеряются - PostgreSQL использует Docker volume, который сохраняется на EBS диске AWS
>
> **План восстановления:**
> 1. **Автоматическое восстановление сервиса:**
> - Docker Compose настроен с `restart: always` - контейнеры автоматически перезапускаются
> - AWS Health Checks могут автоматически перезагрузить EC2 instance при проблемах
>
> 1. **Восстановление сервера:**
> - У меня есть Terraform конфигурация - одна команда `terraform apply` создаст идентичный сервер
> - Ansible playbook настроит новый сервер за 5-10 минут
> - Бэкапы базы данных хранятся отдельно (AWS snapshots)
>
> **Улучшения для production:**
> - **Multi-AZ deployment** - серверы в разных зонах доступности
> - **Load Balancer** - распределение трафика между несколькими инстансами
> - **Auto Scaling Group** - автоматическое создание новых серверов при проблемах
> - **Database replication** - master-slave PostgreSQL
> - **Automated backups** - ежедневные snapshots с retention policy
> - **Monitoring alerts** - немедленное уведомление при downtime
>
> С Kubernetes это решается проще - он автоматически перезапускает failed pods на других нодах.”

Q4: Как можно масштабировать ваше приложение при росте нагрузки?

Ответ:

> “Архитектура спроектирована с учётом масштабирования. Вот варианты:
>
> **Вертикальное масштабирование (Scale Up):**
> - Увеличить размер EC2 instance (с t2.medium на t2.large)
> - Одно изменение в Terraform: `instance_type = "t2.large"`
> - Просто, но есть лимит
>
> **Горизонтальное масштабирование (Scale Out) - лучший подход:**
>
> **1. Application tier:**
> - Поднять несколько реплик Flask приложения
> - В Docker Compose: `docker-compose up --scale web=3` - 3 экземпляра приложения
> - В Kubernetes: `kubectl scale deployment flask-app --replicas=5`
> - Добавить Load Balancer (AWS ALB) перед ними
> - Stateless приложение позволяет это легко сделать
>
> **2. Database tier:**
> - PostgreSQL master-slave репликация
> - Read queries идут на slaves, write на master
> - Или перейти на AWS RDS с Multi-AZ
> - Connection pooling (PgBouncer) для эффективного использования соединений
>
> **3. Caching tier:**
> - Redis cluster с несколькими нодами
> - Redis Sentinel для high availability
> - Или AWS ElastiCache
>
> **4. Auto-scaling:**
> - В Kubernetes настроить Horizontal Pod Autoscaler (HPA)
> - Метрика: если CPU > 70%, добавить replica
> - Автоматически адаптируется к нагрузке
>
> **5. CDN:**
> - Статические файлы (CSS, JS, images) через CloudFront
> - Снижает нагрузку на сервер
>
> **Bottlenecks и оптимизация:**
> - Grafana покажет, где bottleneck - CPU, память, database?
> - На основе метрик решаем, что масштабировать
>
> Kubernetes делает всё это тривиальным - меняешь число реплик в манифесте, остальное автоматически.”

Q5: Какие именно метрики вы собираете и почему они важны?

Ответ:

> “Я собираю метрики на трёх уровнях:
>
> **1. Infrastructure Metrics (Node Exporter):**
> - **CPU usage** - показывает загрузку процессора. Если >80%, нужно масштабирование
> - **Memory usage** - отслеживаю RAM. Swap activity - плохой знак
> - **Disk I/O** - читает/записывает ли диск активно. Помогает оптимизировать БД
> - **Network traffic** - входящий/исходящий трафик. Помогает обнаружить DDoS или утечки
> - **Disk space** - предотвращает заполнение диска, которое уронит сервисы
>
> **2. Application Metrics (Flask metrics через prometheus_client):**
> - **Request rate** - сколько запросов в секунду приходит
> - **Response time** - latency каждого endpoint. Если >500ms, нужна оптимизация
> - **Error rate** - процент 4xx и 5xx ошибок. Spike означает проблему
> - **Requests by endpoint** - какие URL наиболее популярны
> - **Active users/sessions** - сколько пользователей сейчас онлайн
>
> **3. Container Metrics (cAdvisor):**
> - **Container health** - running/stopped status
> - **Container resource usage** - CPU/Memory per container
> - **Restart count** - если контейнер часто рестартует, есть проблема
>
> **4. Database Metrics (PostgreSQL Exporter):**
> - **Connection count** - сколько активных соединений
> - **Query performance** - slow queries detection
> - **Database size** - рост данных
> - **Locks** - блокировки таблиц
>
> **Почему это важно:**
> - **Проактивность:** вижу проблемы ДО того, как пользователи пожалуются
> - **Optimization:** метрики показывают, где узкие места
> - **Capacity planning:** тренды помогают планировать апгрейды
> - **Debugging:** когда что-то сломалось, метрики показывают причину
> - **SLA tracking:** могу измерить uptime и availability
>
> Все метрики визуализированы в Grafana с красивыми графиками и alerts.”

Q6: Как вы обеспечиваете безопасность? Что если кто-то попытается взломать?

Ответ:

> “Безопасность - многоуровневая:
>
> **Network Level:**
> - **AWS Security Groups** - whitelist только нужных портов. SSH (22), HTTP (80), Grafana (3000), Prometheus (9090). Всё остальное закрыто
> - **Firewall (UFW)** - дополнительный слой на самом сервере
> - **Fail2Ban** - если кто-то пытается bruteforce SSH, автоматический ban IP после 3 попыток
> - **SSH keys only** - пароли полностью отключены. Только публичный ключ
> - **Non-standard SSH port** (можно настроить, сейчас стандартный 22)
>
> **Application Level:**
> - **No hardcoded secrets** - все пароли в environment variables
> - **SQL Injection protection** - использую SQLAlchemy ORM, который параметризует запросы
> - **XSS protection** - Flask автоматически escapes HTML
> - **CSRF tokens** - защита форм от cross-site request forgery
> - **Input validation** - проверка всех пользовательских данных
> - **Rate limiting** (можно добавить Flask-Limiter) - защита от DDoS
>
> **Container Level:**
> - **Non-root users** - контейнеры не работают от root
> - **Read-only filesystem** где возможно
> - **Image scanning** - можно добавить Trivy/Clair для поиска CVE уязвимостей
> - **Minimal base images** - использую python:3.9-slim, не full Ubuntu
> - **No sensitive data in images** - secrets передаются через environment
>
> **Data Level:**
> - **Encrypted secrets** - GitHub Secrets encrypted at rest
> - **Database backups** - шифрованные AWS snapshots
> - **Secrets management** - Kubernetes Secrets (base64, можно улучшить до Vault)
> - **TLS/SSL** - следующий шаг: Let’s Encrypt для HTTPS
>
> **Access Control:**
> - **Principle of least privilege** - каждый сервис имеет минимальные необходимые права
> - **AWS IAM roles** - вместо хранения AWS credentials
> - **Audit logging** - все действия логируются
>
> **Monitoring & Response:**
> - **Log analysis** - мониторинг подозрительной активности
> - **Alerts** - уведомления при аномалиях
> - **Incident response plan** - что делать при взломе
>
> **Что можно улучшить:**
> - HTTPS с автоматическим обновлением сертификатов
> - WAF (Web Application Firewall)
> - HashiCorp Vault для управления секретами
> - Регулярные security audits и penetration testing
> - SIEM система для централизованного анализа безопасности”

Q7: С какими самыми сложными проблемами вы столкнулись и как их решили?

Ответ:

> “Самая сложная проблема была связана с **памятью и стабильностью**.
>
> **Проблема:**
> Приложение периодически падало. В Grafana я увидел, что память растёт до 100%, затем OOM Killer убивает процессы.
>
> **Debugging процесс:**
> 1. **Проверил логи:** `docker logs flask-app` показал Out of Memory errors
> 2. **Анализ метрик:** Grafana показала, что память растёт линейно
> 3. **Профилирование:** Добавил memory_profiler в код
> 4. **Обнаружил:** Утечка памяти из-за неправильного кэширования + слишком много Gunicorn workers
>
> **Решение:**
> 1. **Resource limits в Docker Compose:**
> `yaml`
> `  deploy:`
> `    resources:`
> `      limits:`
> `        memory: 512M`
> `      reservations:`
> `        memory: 256M`
> Это предотвращает один контейнер от захвата всей памяти
>
> 1. **Оптимизация Gunicorn:**
> - Было: 8 workers (слишком много для t2.medium)
> - Стало: 4 workers + threads
> - Formula: `(2 x CPU cores) + 1`
>
> 2. **Redis для session management:**
> - Вместо хранения сессий в памяти Flask
> - Redis хранит их отдельно
>
> 3. **Swap настройка:**
> - Добавил 2GB swap на сервере
> - Помогает при временных спайках
>
> 4. **Database connection pooling:**
> - SQLAlchemy pool size ограничен
> - Предотвращает утечку соединений
>
> 5. **Monitoring alerts:**
> - Настроил алерт при memory >85%
> - Получаю уведомление до критической ситуации
>
> **Результат:**
> - Memory usage стабилизировалась на 60-70%
> - Нет больше OOM crashes
> - Приложение работает стабильно неделями
>
> **Что я узнал:**
> - Importance of monitoring - без Grafana я бы не нашёл причину
> - Resource limits - ВСЕГДА устанавливайте лимиты в production

> - Profiling - не гадать, а измерять
> - Documentation - я всё задокументировал для будущего
>
> Эта проблема научила меня больше, чем если бы всё работало сразу.”

💡 5. СОВЕТЫ ПО ПРЕЗЕНТАЦИИ

Как держаться:

- ✅ Говорите уверенно - вы эксперт в своём проекте
- ✅ Смотрите на аудиторию, не только на экран
- ✅ Используйте жесты для emphasize
- ✅ Улыбайтесь - показывайте enthusiasm о проекте
- ✅ Говорите в темпе - не слишком быстро, не слишком медленно
- ❌ Не извиняйтесь за “мелкие недочёты” - фокус на достижениях
- ❌ Не говорите “я попытался” - говорите “я реализовал”
- ❌ Не стойте неподвижно - двигайтесь естественно
- ❌ Не читайте со слайдов дословно

Длительность:

- **Базовая версия:** 12-15 минут
- **Короткая версия:** 7-10 минут (если времени мало)
- **Расширенная версия:** 20-25 минут (с подробной демонстрацией)

Если забыли что сказать:

- Посмотрите на слайд - он напомнит структуру
- Скажите: “Позвольте показать это в действии” - переключитесь на demo
- Сделайте паузу, соберитесь с мыслями - это нормально
- Можете честно сказать: “Давайте я покажу это на примере”

Как отвечать на вопросы:

- Слушайте вопрос полностью, не перебивайте
- Если не поняли - переспросите
- Если не знаете ответ - честно скажите: “Хороший вопрос, я это ещё не изучал, но вот что я думаю...”
- Отвечайте кратко и по сути
- Можете показать на экране вместо объяснения словами
- Благодарите за интересные вопросы

Что показать в первую очередь (priority):

1. **Работающее приложение** - доказывает, что проект реален
2. **Grafana** - впечатляет визуально, показывает профессионализм
3. **GitHub Actions** - демонстрирует автоматизацию
4. **Docker containers на сервере** - технические детали
5. **Infrastructure code** - подтверждает IaC подход

Энергия и энтузиазм:

- Говорите о проекте с гордостью - вы проделали большую работу!
- Подчёркивайте, что именно вас больше всего excited
- Делитесь challenges с улыбкой - они сделали вас сильнее
- Показывайте passion к DevOps

✅ 6. ЧЕК-ЛИСТ ПЕРЕД ПРЕЗЕНТАЦИЕЙ

За день до презентации:

Проверить работоспособность:
- [ ] Приложение доступно: http://3.127.155.114
- [ ] Grafana работает: http://3.127.155.114:3000
- [ ] Prometheus собирает метрики: http://3.127.155.114:9090
- [ ] Все Docker контейнеры запущены: `docker ps` показывает 7 контейнеров
- [ ] Нет critical alerts в Grafana
- [ ] GitHub Actions последний run успешен (зелёная галочка)
- [ ] База данных содержит демо-данные (не пустая)

Подготовить credentials:
- [ ] Grafana login/password готовы (admin/ваш_пароль)
- [ ] SSH ключ для доступа к серверу работает
- [ ] GitHub аккаунт залогинен

Подготовить браузер:
- [ ] Открыть вкладки заранее:
- Приложение: http://3.127.155.114
- Grafana: http://3.127.155.114:3000
- GitHub: https://github.com/zaburdaev/my-devops-project
- GitHub Actions: https://github.com/zaburdaev/my-devops-project/actions
- [ ] Очистить историю/cache если нужно
- [ ] Закрыть ненужные вкладки
- [ ] Проверить, что нет embarrassing bookmarks на панели

Технические детали:
- [ ] Интернет стабилен
- [ ] AWS сервер не будет shutdown (проверить billing)
- [ ] Docker Hub credentials действительны
- [ ] Достаточно памяти/CPU на сервере

Backup план:
- [ ] Скриншоты всех важных экранов (если упадёт сервер)
- [ ] Локальная копия репозитория
- [ ] Запись экрана работающего приложения (plan B)

За час до презентации:

- [ ] Перезапустить Docker Compose для свежего старта: `docker-compose restart`
- [ ] Проверить логи на ошибки: `docker-compose logs --tail=50`
- [ ] Открыть презентацию

- [ ] Открыть этот скрипт для reference
- [ ] Попить воды
- [ ] Сходить в туалет 😊
- [ ] Глубокий вдох - вы готовы!

Прямо перед презентацией:

- [ ] Проектор/экран подключен корректно
- [ ] Звук не нужен (или работает если нужен)
- [ ] Уведомления на компьютере отключены (Do Not Disturb mode)
- [ ] Телефон на беззвучном
- [ ] Презентация открыта на первом слайде
- [ ] Вода рядом
- [ ] Уверенная улыбка 😊

⏱️ 7. ВАРИАНТЫ ПО ВРЕМЕНИ

КОРОТКАЯ ВЕРСИЯ (7-10 минут)

Если времени мало, фокус на главном:

1. **Введение** (30 сек)
2. **Слайд 1-2:** Титульный + Обзор проекта (1 мин)
3. **Слайд 3:** Архитектура - быстро (1 мин)
4. **Слайд 5:** CI/CD - ключевое достижение (1.5 мин)
5. **Слайд 9:** Мониторинг (1 мин)
6. **DEMO:** Показать работающее приложение + Grafana (2 мин)
7. **Слайд 12:** Заключение (1 мин)

Что пропустить:
- Детали Docker, Ansible, Terraform (упомянуть кратко)
- Kubernetes (или совсем кратко)
- Проблемы и решения
- Безопасность (кратко в заключении)

Что говорить:

> “У нас ограниченное время, поэтому я сфокусируюсь на ключевых достижениях: автоматизации CI/CD и мониторинге. Остальные детали можно обсудить в вопросах.”

РАСШИРЕННАЯ ВЕРСИЯ (20-25 минут)

Если времени много, добавить:

1. **Все 12 слайдов** полностью (12-15 мин)
2. **Расширенная DEMO:** (7-10 мин)
   - Показать приложение подробно
   - Grafana - несколько дашбордов
   - GitHub Actions - показать workflow файл

   - SSH на сервер - показать команды docker, логи
   - Открыть Terraform код
   - Показать Ansible playbook
   - Показать Kubernetes манифесты
3. **Live coding demo** (если уместно):
   - Сделать небольшое изменение в коде
   - Commit + Push
   - Показать как запускается CI/CD
   - Показать как приложение обновляется
4. **Deep dive в один компонент** на выбор:
   - Подробно разобрать CI/CD workflow файл
   - Показать Prometheus queries
   - Объяснить Terraform modules
   - Показать Kubernetes architecture

Дополнительные темы:
- Сравнение до/после DevOps практик
- Cost analysis - сколько стоит инфраструктура
- Performance benchmarks
- Future roadmap подробнее
- Lessons learned расширенно

🎯 ФИНАЛЬНЫЕ НАПУТСТВИЯ

Помните:

1. **Вы - эксперт своего проекта.** Никто не знает его лучше вас.
2. **Это ваше достижение.** Вы прошли путь от нуля до production-ready системы. Гордитесь этим!
3. **Технические проблемы - это нормально.** Если что-то упадёт во время demo, это возможность показать troubleshooting навыки.
4. **Enthusiasm заразителен.** Если вы excited о проекте, аудитория тоже будет.
5. **Вопросы - это хорошо.** Они показывают интерес. “Я не знаю” - приемлемый ответ, добавьте “но я изучу это”.
6. **Практика.** Прорепетируйте хотя бы 2-3 раза перед настоящей презентацией.
7. **Дышите.** Если нервничаете - глубокий вдох, пауза, продолжайте.
8. **Получайте удовольствие!** Это ваш момент показать, чему научились.

Мантра перед презентацией:

> “Я построил полноценную DevOps инфраструктуру.
> Я автоматизировал CI/CD pipeline.
> Я развернул приложение в облаке.
> Я настроил мониторинг и безопасность.
> Я готов поделиться этим с миром.
> Я готов!”

🚀 УДАЧИ!

Вы проделали огромную работу. Теперь время показать её миру!

Useful commands для быстрого reference:

```
# Проверить статус Docker
docker ps
docker-compose ps
docker stats --no-stream

# Посмотреть логи
docker-compose logs -f web
docker-compose logs --tail=100

# Перезапустить сервисы
docker-compose restart
docker-compose up -d

# SSH на сервер
ssh ubuntu@3.127.155.114

# Проверить GitHub Actions статус
gh run list  # если установлен GitHub CLI

# Проверить ресурсы системы
htop
free -h
df -h
```

Контакты проекта:
- 🌐 **Live Demo:** http://3.127.155.114
- 📊 **Grafana:** http://3.127.155.114:3000
- 🔥 **Prometheus:** http://3.127.155.114:9090
- 💻 **GitHub:** https://github.com/zaburdaev/my-devops-project
- 👨‍💻 **Автор:** Vitalii Zaburdaiev
- 🎓 **Курс:** DevOpsUA6

P.S. Этот скрипт - ваш guide, не обязательно следовать дословно. Адаптируйте под свой стиль и ситуацию. Главное - уверенность и знание проекта. Вы справитесь! 💪